

BOOTCAMP MLOps



BOOTCAMP MLOps:

What we will learn to day:
First Cloud Deployment

Danh Pham

Goals

- Know the role of a Container Registry (like Docker Hub)
- Know the differences between Virtual Machines (VMs) and Serverless platforms
- Know how to deploy a container to Google Cloud Run

Contents

Docker and Container Registry recap

Cloud Compute Engine basic concepts

Hands-on Lab

Google Cloud Platform basic concepts

Deploy container to Cloud Run

COLE

Docker and Container Registry recap

Recap & The Big Question

- We built a Docker image.
- This image packages our API code, Python libraries, and all dependencies.
- It solves the "it works on my machine" problem.

The Big Question: How do we run this image so the world can access it 24/7?



COLE

Cloud Compute Engine basic concepts

Recap & The Big Question

We need to "rent" computer power from a cloud provider (like Google, Amazon, or Microsoft).
There are two main ways to do this:
Virtual Machines (VMs)
Serverless

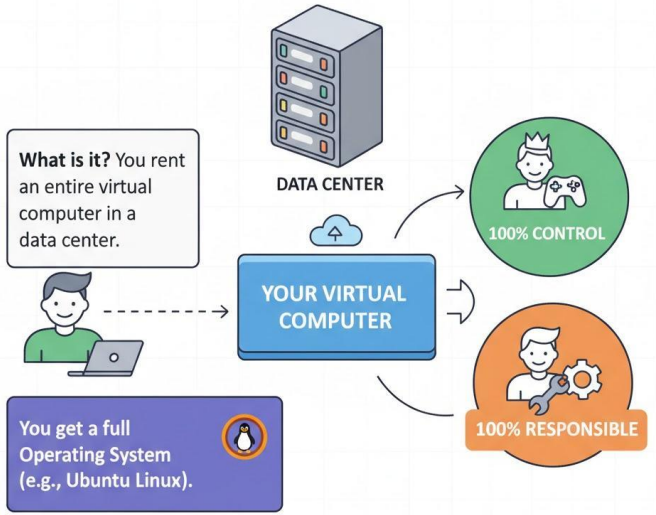


COLE

Option 1: The "Old" Way - Virtual Machine (VM)

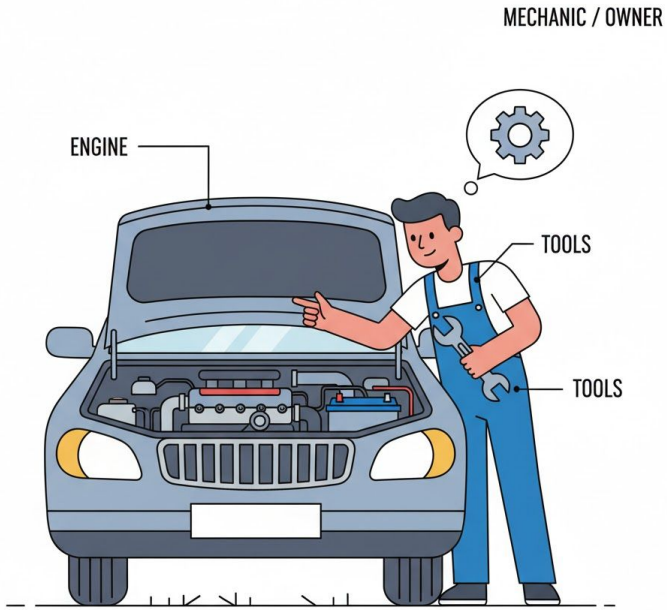
What is it?

- You rent an entire virtual computer in a data center.
- You get a full Operating System (e.g., Ubuntu Linux).
- You have 100% control.
- You are 100% responsible.



VM Analogy: Owning a Car

- You have total control (you can paint it, tune the engine).
- You are responsible for everything:
  - Gas (paying for it 24/7)
  - Maintenance (OS patching)
  - Insurance (Security)
  - Fixing it when it breaks



Key VM Products



Google Compute Engine



Azure Virtual Machines



Amazon EC2

Option 2: The "New" Way - Serverless Containers

What is it?

- You only provide your container image.
- The platform runs it for you.
- You **do not** see, manage, or patch the Operating System.
- It automatically scales up and down (even to $0).



Serverless Analogy: Using a Ride-Sharing App

- You don't own the car. You don't care about its maintenance.
- You just say, "I need a ride from A to B."
- You **pay only for the ride** you take.
- If you have 100 friends, you can call 100 rides instantly.

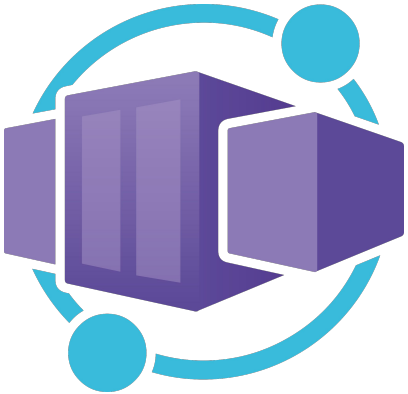


COLE

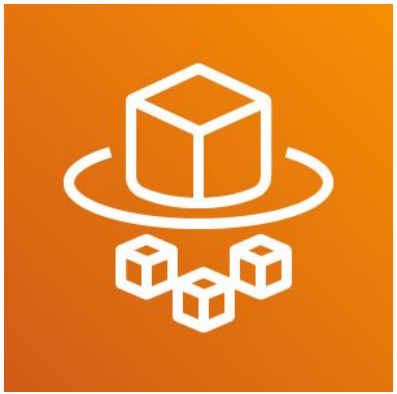
Key Serverless Container Products



Google Cloud Run



Azure Container Apps



AWS Fargate

VM vs. Serverless: Which is Better?

Feature	Virtual Machine (VM)	Serverless (Cloud Run)
Management	You manage the OS, security, patching, runtime, dependencies, etc.	**Google manages everything** (OS, scaling, patching, infrastructure).
Scaling	**Manual** (You typically add or remove VMs manually or configure complex auto-scaling groups).	**Auto-scaling** (Scales automatically and instantly from zero to thousands of instances based on traffic).
Cost	**Pay 24/7** (You pay for the VM's reserved resources, even when it's idle).	**Pay-per-use** (Only pay when your code is processing a request, scales down to $0 when idle).
Our API	(Too much work for a simple API—requires high overhead for low traffic).	**Perfect Choice!** (Ideal for simple, stateless APIs with variable or low traffic).

FEATURE	BENEFIT
Speed	Faster Delivery
Cost Efficiency	Reduced Expenses
Scalability	Growth Potential

COMPARISON TABLE

COLE

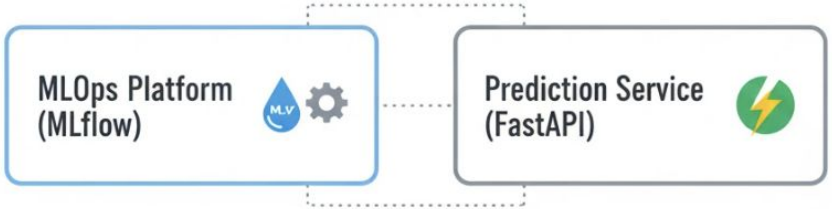
The Big "But"... What About Database?

- Wait... Serverless is **STATELESS.**
- But in Session 2, our MLflow server was **STATEFUL.**
- It needed a database and a file system to save models.
- **Problem:** We can't run the Database server on Cloud Run.



The MLOps Secret: You Have TWO Services

- You don't have one application. You have two different services with two different jobs.
- **Service 1:** The MLOps Platform (MLflow, Airflow, …)
- **Service 2:** The Prediction Service (FastAPI)
- **The Goal:** Use the right tool for the right job.



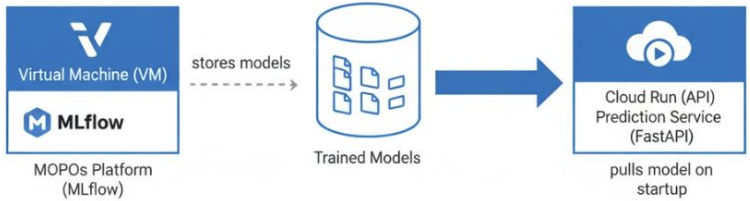
Service 1 vs. Service 2: The Right Tool for the Job

Feature	1. MLOps Platform (MLflow)	2. Prediction Service (FastAPI)
Primary Job	**Track experiments**, store models, manage model registry.	**Serve predictions** (real-time, low-latency API endpoint).
Traffic Pattern	**Low** (Steady usage by a few users—your ML team and CI/CD).	**Spiky** (users—traffic based on end-user demand).
State	**Stateful** (Needs a **database** to persistently store experiment data and model metadata).	**Stateless** (No session memory; the only "state" is the loaded model, which is read-only).
Best Tool	**VM or Managed Database** (Needs a stable, always-on resource with persistent storage).	**Serverless (Cloud Run)** (Ideal for auto-scaling from zero and pay-per-use costing).

COLE

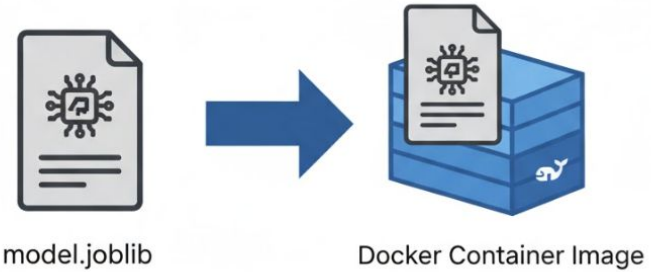
The "Pro" Way: The Dynamic Model

- This is the full, professional MLOps architecture.
- The MLflow server runs on its own **VM** or as least MLFlow use Database running on a **VM**.
- The model.joblib files are saved in **Cloud Storage (GCS).**
- Our Cloud Run API, on startup, *downloads* the "Production" model from the registry.



Our Lab Today: The "Baked-In" Model

- The "Pro" way is too complex for today's 70-minute lab.
- We will use a simpler, common, and valid approach.
- We will "**bake**" the model.joblib file *directly* into our Docker image.
- The model will be a local file, just like api.py.



"Baked-In" vs. "Dynamic": The Trade-Off

- **Baked-In (Today's Lab)**
  - **Pro:** Simple, fast, no external dependencies.
  - **Con:** To update the model, you must **re-build and re-deploy** the entire image.
- **Dynamic (Session 10)**
  - **Pro:** Can update the model *without* a new deployment.
  - **Con:** More complex, more moving



1. Model in Docker Image

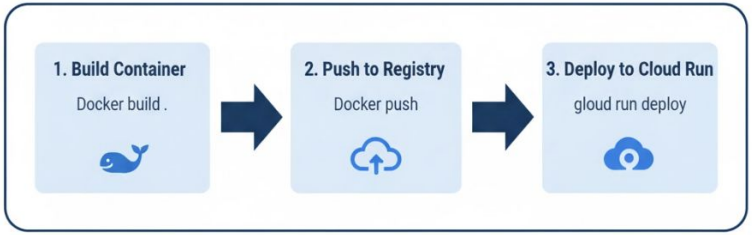


2. Model Pulled at Runtime

	Container Size	
1	Large	Small
2	Build Time Slow	Fast
3	2. Model Updates (Rebuild + Deploy)	Fast (Restart Container)

Our Deployment Workflow (The Big Picture)

- **Build (Locally):** docker build
  - (We add the model.joblib to the Dockerfile)
- **Push (To Registry):** docker push
  - (Uploads our image to Docker Hub)
- **Deploy (From Registry):** Google Cloud Run
  - (Pulls the image and runs it)



Key Tools for the Lab



Docker Hub

- Our **Container Registry**.
- The "GitHub for Docker Images."
- The central "garage" where we store our built images.



Google Cloud Run

- Our **Serverless Platform**.
- The "ride-share" service that runs our image.

COLE

Hands-on Lab

Let's Deploy!

- **Final Checklist:**
  1. Do you have a Docker Hub account?
  2. Do you have a Google Cloud (GCP) account?
  3. Have you activated your free credits?
  4. Do you have your model.joblib file ready?
- **Questions?**
- **Next:** Hands-On Lab

COLE

Your mission

1. **Create Google Cloud Platform account**: Ensure you have a GCP account (pre-lab requirement).
2. **Build a docker image with bake-in model:** Train your model first and add model to your image.
3. **Create a Docker Hub Account:** Sign up for a free account and push the image.
4. **Deploy a Cloud Run service:** Deploy image and test.



Q&A



Thank you for listening!

